Three Pre-Tiling Practice Exams

Designed to match the style and reasoning level of the course practice midterm. Complete all three tests before turning to the solution section.

SCOPE AND STUDY METHOD

Included: Amdahl/Gustafson/Roofline, decomposition and collectives, CPU parallelism, coherence and consistency, false sharing, GPU hierarchy and SIMT, CUDA indexing, coalescing, memory spaces, barriers, atomics, OpenMP, and MPI. Excluded: tiling algorithms and occupancy calculations.

MOCK TEST 1

EXAM CONDITIONS

60 minutes · 100 points · Closed notes unless your instructor states otherwise. Show formulas, substitutions, and the first few lane/thread indices for mapping questions. Assume a warp has 32 threads unless stated otherwise.

Question 1 — True / False [14 points]

	Statement	Answer
(a)	Amdahl's law assumes a fixed total workload while processor count changes.	T / F
(b)	If the serial fraction is zero, Gustafson's scaled speedup equals the processor count.	T / F
(c)	Parallel efficiency is speedup divided by the number of processors.	T / F
(d)	Operational intensity is measured in floating-point operations per byte transferred.	T / F
(e)	Increasing operational intensity always increases attainable performance without limit.	T / F
(f)	Cache coherence defines the global ordering rules for accesses to all memory locations.	T / F
(g)	False sharing can occur when two threads update different words in the same cache line.	T / F
(h)	A CUDA programmer creates warps explicitly in kernel code.	T / F
(i)	A branch causes no warp divergence when every active lane takes the same path.	T / F
(j)	CUDA shared memory is allocated per thread block.	T / F
(k)	A barrier and an atomic operation provide the same guarantee.	T / F
(l)	Memory allocated by cudaMallocManaged() can be accessed by both the CPU and GPU.	T / F
(m)	GPU cores favor throughput by devoting less hardware to complex single-thread control.	T / F
(n)	CUDA blocks are guaranteed to execute in increasing blockIdx.x order.	T / F

Question 2 — Define briefly [12 points]

- (a) Domain decomposition
- (b) Memory consistency
- (c) Warp divergence

Question 3 — Amdahl's law [12 points]

A program has a serial fraction of 0.15. Compute (a) its speedup on 12 processors and (b) its theoretical maximum speedup as the processor count approaches infinity.

Question 4 — Gustafson's law [10 points]

For a scaled workload running on 16 processors, the measured serial fraction is 0.08. Compute the scaled speedup.

Question 5 — CUDA 1D launch and bounds [12 points]

A vector contains n = 10,000 elements and the block size is 256 threads. Write (a) the global index expression, (b) the grid-size expression and numerical block count, (c) the bounds check, and (d) the number of launched threads that do no useful element work.

Question 6 — Three outputs per thread [10 points]

Each thread computes three vector-addition outputs. B = blockDim.x. Which initialization makes the first access of neighboring threads coalesced and gives each block a disjoint 3B-element region?

```
(a) int i = 3 * blockIdx.x * blockDim.x + threadIdx.x;
(b) int i = 3 * (blockIdx.x * blockDim.x + threadIdx.x);
(c) int i = 3 * threadIdx.x * blockDim.x + blockIdx.x;
(d) int i = (threadIdx.x * blockDim.x + blockIdx.x) * 3;
```

For the correct choice, write the other two indices processed by the thread and list the first indices generated by lanes 0–3.

Question 7 — CUDA 2D mapping [12 points]

Write row, column, and row-major linear-index expressions for an H × W matrix when blockDim = (32, 8). For H = 513 and W = 1000, give grid.x and grid.y and write the complete bounds condition.

Question 8 — Warp divergence [10 points]

A block has 64 threads and executes the code below. Identify which warp(s) diverge and state the active lanes on each branch path.

```
if (threadIdx.x < 20) {
    y[threadIdx.x] = 2.0f * x[threadIdx.x];
} else {
    y[threadIdx.x] = x[threadIdx.x];
}
```

Question 9 — Shared-memory instances [8 points]

A kernel launches 600 blocks, and each block declares `__shared__ float s[256];`. How many logical instances of s exist over the kernel launch? Who can access one instance? Is its lifetime longer than the block?

WORK SPACE — continue calculations or code here

MOCK TEST 2

EXAM CONDITIONS

60 minutes · 100 points · Closed notes unless your instructor states otherwise. Show formulas, substitutions, and the first few lane/thread indices for mapping questions. Assume a warp has 32 threads unless stated otherwise.

Question 1 — True / False [14 points]

	Statement	Answer
(a)	Parallel efficiency is defined as S(p)/p.	T / F
(b)	Gustafson's law models a fixed-size workload.	T / F
(c)	MPI Allreduce returns the reduced result to every participating rank.	T / F
(d)	A superscalar processor may issue more than one instruction in a cycle.	T / F
(e)	Out-of-order execution is allowed to violate true data dependencies.	T / F
(f)	SIMD lanes normally execute unrelated instructions in the same vector instruction.	T / F
(g)	Coherence concerns how writes to one memory location become visible to processors.	T / F
(h)	Consistency specifies permitted ordering and visibility across memory operations.	T / F
(i)	False sharing necessarily means that the source program has a data race.	T / F
(j)	Different CUDA blocks may be scheduled in any order.	T / F
(k)	The kernel chooses the hardware warp width at launch time.	T / F
(l)	Constant memory is especially effective when many lanes read the same location.	T / F
(m)	__syncthreads() is a grid-wide barrier.	T / F
(n)	Shared memory is visible to threads in the same block, not to other blocks.	T / F

Question 2 — Define briefly [12 points]

- (a) Simultaneous multithreading (SMT)
- (b) Allreduce
- (c) False sharing

Question 3 — Roofline model [12 points]

A GPU has peak compute performance P_{peak} = 12 TFLOP/s and memory bandwidth B = 600 GB/s. A kernel has operational intensity I = 8 FLOP/byte. Compute (a) the bandwidth roof, (b) attainable performance under the Roofline model, (c) whether the kernel is compute- or memory-bound, and (d) the ridge-point intensity.

Question 4 — Infer the serial fraction [10 points]

A fixed-size program achieves a speedup of 5.0 on 8 processors. Assuming Amdahl's law, estimate the serial fraction and the maximum possible speedup.

Question 5 — Decomposition and collectives [12 points]

A dense matrix A is decomposed by rows across MPI ranks. Rank 0 initially owns the full input vector x. Each rank computes its local rows of y = Ax.

- (a) Which collective distributes x to every rank?
- (b) Which collective should be used if only rank 0 needs the complete y?
- (c) Which collective should be used if every rank needs the complete y?
- (d) State one advantage of row decomposition for this operation.

Question 6 — Coalescing a row-major matrix [10 points]

For a row-major matrix M with W columns, compare these two mappings for a 2D block. Which one provides unit-stride accesses across neighboring threadIdx.x lanes, and why?

```
Mapping A: row = blockIdx.y * blockDim.y + threadIdx.y;
           col = blockIdx.x * blockDim.x + threadIdx.x;

Mapping B: row = blockIdx.y * blockDim.y + threadIdx.x;
           col = blockIdx.x * blockDim.x + threadIdx.y;

Access:   M[row * W + col]
```

Question 7 — Shared-memory synchronization [12 points]

The kernel below is intended to copy the value loaded by the next thread in the block. Identify the correctness problem and repair it. Assume blockDim.x is a multiple of 32 and one block processes one independent segment.

```
__shared__ float s[256];
int t = threadIdx.x;
int i = blockIdx.x * blockDim.x + t;
s[t] = in[i];
out[i] = s[(t + 1) % blockDim.x];
```

Question 8 — Grid-wide synchronization [10 points]

A kernel writes partial results in every block and then wants block 0 to consume all of them later in the same ordinary kernel. Can __syncthreads() make this correct? Give a safe implementation strategy.

Question 9 — Race and reduction [8 points]

Why is the following device code incorrect when executed by many threads? Give two valid repair strategies and state which usually scales better for a large reduction.

```
sum[0] += A[i];
```

WORK SPACE — continue calculations or code here

MOCK TEST 3

EXAM CONDITIONS

60 minutes · 100 points · Closed notes unless your instructor states otherwise. Show formulas, substitutions, and the first few lane/thread indices for mapping questions. Assume a warp has 32 threads unless stated otherwise.

Question 1 — True / False [14 points]

	Statement	Answer
(a)	Speedup is T(1)/T(p).	T / F
(b)	Parallel overhead may be expressed as pT(p) - T(1).	T / F
(c)	A Roofline kernel is bandwidth-bound when B × I is below peak compute.	T / F
(d)	Dynamic scheduling always has less scheduling overhead than static scheduling.	T / F
(e)	MPI ranks normally share one address space.	T / F
(f)	OpenMP threads in one process normally share the process address space.	T / F
(g)	CUDA local memory is guaranteed to reside in fast on-chip storage.	T / F
(h)	Unified Memory eliminates all migration and page-fault costs.	T / F
(i)	Threads in different warps cause warp divergence when they take different branches.	T / F
(j)	A warp's active mask selects which lanes act on the current instruction.	T / F
(k)	Independent CUDA blocks may execute concurrently or in different orders.	T / F
(l)	An atomic addition acts as a barrier for all threads in the block.	T / F
(m)	A wavefront is AMD terminology for a hardware group analogous to a warp.	T / F
(n)	Coalescing concerns the addresses requested together by neighboring lanes.	T / F

Question 2 — Define briefly [12 points]

- (a) Operational intensity
- (b) All-gather
- (c) GPU texture cache/path

Question 3 — Speedup, efficiency, overhead [12 points]

A program takes 120 s on one processor and 20 s on 8 processors. Compute speedup, efficiency, and parallel overhead pT(p) - T(1).

Question 4 — Scheduling irregular work [10 points]

An OpenMP loop processes graph vertices whose degrees vary from 1 to 100,000. Choose static or dynamic scheduling, explain the load-balance/overhead tradeoff, and state how chunk size affects that tradeoff.

Question 5 — Matrix-addition launch [12 points]

Matrices have 1000 rows and 750 columns. A CUDA kernel uses blockDim = (16, 16). Write (a) row and column expressions, (b) grid dimensions, (c) the bounds condition, and (d) the row-major index.

Question 6 — Two adjacent outputs per thread [10 points]

Each thread computes two adjacent vector outputs. Variable i is the first index handled by the thread. Which expression gives correct, coalesced accesses to the first elements?

```
(a) int i = (blockIdx.x * blockDim.x) * 2 + threadIdx.x;
(b) int i = (blockIdx.x * blockDim.x + threadIdx.x) * 2;
(c) int i = (threadIdx.x * blockDim.x) * 2 + blockIdx.x;
(d) int i = (threadIdx.x * blockDim.x + blockIdx.x) * 2;
```

For the correct choice, write the second index and compare the first four lane addresses with option (b).

Question 7 — Divergence by lane pattern [12 points]

Within one 32-thread warp, the branch condition is `threadIdx.x % 4 == 0`. How many lanes execute the true path? List them. How many execute the false path? Does the warp diverge? Explain how the two paths execute.

Question 8 — Coherence and false sharing [10 points]

Two CPU threads repeatedly increment counters[0] and counters[1]. Each thread owns one counter, so there is no logical data race. The counters occupy the same cache line and performance is poor. Explain the hardware behavior and give two fixes.

Question 9 — OpenMP and MPI correctness [8 points]

- (a) What OpenMP clause repairs a parallel loop that adds each element into scalar sum?

- (b) Every MPI rank owns a different vector segment, and every rank must receive the concatenated full vector. Which collective is required?

WORK SPACE — continue calculations or code here

SOLUTIONS START HERE

CHECK YOUR WORK, NOT JUST THE LETTER

For every numerical problem, verify the model, units, substitution, and conclusion. For CUDA mapping, trace lanes 0–3. For synchronization, name both the participants and the memory ordering requirement.

Quick answer key

Set	True/False	Key result 1	Key result 2	Mapping
Test 1	Q1: T T T T F F T F T T F T T F	Q3 4.53×; limit 6.67×	Q4 14.8	Q6 (a)
Test 2	Q1: T F T T F F T T F T F T F T	Q3 4.8 TFLOP/s, memory-bound	Q4 s ≈ 0.0857	Q6 A
Test 3	Q1: T T T F F T F F F T T F T T	Q3 6×, 75%, 40 s	Q5 grid (47,63)	Q6 (a)

MOCK TEST 1 — WORKED SOLUTIONS

Question 1 — True / False

	Ans.	Reason
(a)	T	Amdahl holds total problem size fixed.
(b)	T	Gustafson: S = p - (p-1)s; if s = 0, S = p.
(c)	T	E(p) = S(p)/p.
(d)	T	I = operations / bytes moved.
(e)	F	Performance stops rising at the compute roof.
(f)	F	That is consistency. Coherence focuses on values/order for one location.
(g)	T	Different words can still invalidate the same cache line.
(h)	F	Hardware forms warps from consecutive thread IDs.
(i)	T	Uniform control flow keeps all active lanes on one path.
(j)	T	Each block receives its own shared-memory instance.
(k)	F	A barrier waits for participants; an atomic serializes one memory update.
(l)	T	Unified Memory is accessible from host and device, with runtime migration.
(m)	T	GPU design emphasizes throughput and latency hiding.
(n)	F	Blocks are independent and have no guaranteed order.

Question 2 — Definitions

2(a) Domain decomposition

Partition the data domain into pieces, assign pieces to workers, and compute mostly on local data. Examples include matrix rows, image regions, or vector segments.

2(b) Memory consistency

The rules that specify the allowed order and visibility of loads and stores across threads and memory locations. Coherence answers a narrower one-location question.

2(c) Warp divergence

A warp diverges when its active lanes choose different control-flow paths. The paths are executed separately under different active masks, reducing lane utilization.

Question 3 — Amdahl's law

Use S(p) = 1 / [s + (1-s)/p].

S(12) = 1 / [0.15 + 0.85/12] = 1 / 0.220833... = 4.53×.

As p → ∞, (1-s)/p → 0, so S_{max} = 1/s = 1/0.15 = 6.67×.

Question 4 — Gustafson's law

$S_G(p) = p - (p-1)s = 16 - 15(0.08) = 16 - 1.2 = 14.8×.$

Question 5 — CUDA 1D launch

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
int blocks = (n + blockDim.x - 1) / blockDim.x;  // or (n + 255) / 256
if (i < n) { C[i] = A[i] + B[i]; }
```

blocks = ceil(10000/256) = 40. Launched threads = 40 × 256 = 10,240, so 240 threads fail the bounds check.

Question 6 — Three outputs per thread

Correct: (a). It assigns block k the region [3kB, 3(k+1)B - 1] while lanes first touch consecutive elements.

```
int i0 = 3 * blockIdx.x * blockDim.x + threadIdx.x;
int i1 = i0 + blockDim.x;
int i2 = i0 + 2 * blockDim.x;
```

Lane	First index i0	Second i1	Third i2
0	3kB	3kB + B	3kB + 2B
1	3kB + 1	3kB + B + 1	3kB + 2B + 1
2	3kB + 2	3kB + B + 2	3kB + 2B + 2
3	3kB + 3	3kB + B + 3	3kB + 2B + 3

Option (b) makes the first addresses 3kB, 3kB+3, 3kB+6, ...: a stride-3 pattern rather than unit stride.

Question 7 — CUDA 2D mapping

```
int col = blockIdx.x * blockDim.x + threadIdx.x;
int row = blockIdx.y * blockDim.y + threadIdx.y;
if (row < H && col < W) {
    int idx = row * W + col;
    // use matrix[idx]
}
```

grid.x = ceil(1000/32) = 32; grid.y = ceil(513/8) = 65. Mapping threadIdx.x to columns also makes neighboring x-lanes access adjacent row-major elements.

Question 8 — Warp divergence

Warp	Threads	True-path lanes	Else-path lanes	Diverges?
0	0–31	0–19	20–31	Yes
1	32–63	None	0–31 of that warp	No

Warp 0 executes both paths under complementary masks. Warp 1 executes only the else path. Divergence is a within-warp property; different warps may take different paths without diverging each other.

Question 9 — Shared-memory instances

There are 600 logical instances, one per block over the launch. Only threads in a block can access that block's instance. Its lifetime is the block's lifetime; it does not persist after the block completes.

MOCK TEST 2 — WORKED SOLUTIONS

Question 1 — True / False

	Ans.	Reason
(a)	T	E = S/p.
(b)	F	Gustafson scales problem size with p.
(c)	T	Every rank receives the reduced value.
(d)	T	Superscalar issue width can exceed one.
(e)	F	Out-of-order hardware must preserve dependencies and architectural behavior.
(f)	F	SIMD lanes apply one vector instruction to multiple data elements.
(g)	T	Coherence is primarily a per-location visibility/order property.
(h)	T	Consistency constrains observable ordering across memory operations.
(i)	F	Different variables can share a line without a source-level race.
(j)	T	Blocks are independent and scheduling order is unspecified.
(k)	F	Warp width is an architectural property.
(l)	T	Uniform constant reads can be broadcast efficiently.
(m)	F	It synchronizes threads in one block only.
(n)	T	Shared memory has block scope.

Question 2 — Definitions

2(a) Simultaneous multithreading

One CPU core keeps state for multiple hardware threads and may issue instructions from more than one of them in a cycle, improving utilization when a thread stalls or has limited instruction-level parallelism.

2(b) Allreduce

Combine values from all ranks with an operation such as sum, maximum, or minimum, then return the reduced result to every rank.

2(c) False sharing

Independent variables used by different cores occupy the same coherence unit/cache line. Writes cause needless invalidations and line transfers even though the threads do not logically share a variable.

Question 3 — Roofline

$$P_{attainable} = \min(P_{peak}, B \times I)$$

B × I = 600 GB/s × 8 FLOP/byte = 4,800 GFLOP/s = 4.8 TFLOP/s.

min(12, 4.8) = 4.8 TFLOP/s, so the kernel is memory-bound. The ridge point is $I^* = P_{peak}/B = 12{,}000/600 = 20$ FLOP/byte.

Question 4 — Infer serial fraction

From $1/S = 1/p + s(1 - 1/p)$:

$$s = (1/S - 1/p) / (1 - 1/p) = (0.2 - 0.125)/0.875 = 0.0857$$

The estimated serial fraction is 8.57%. Therefore $S_{max} = 1/s \approx 11.67\times$.

Question 5 — Decomposition and collectives

Part	Answer	Why
(a)	Broadcast x	The root owns x and every rank needs the same full vector.
(b)	Gather y segments	Only rank 0 needs the assembled result.
(c)	Allgather y segments	Every rank needs the concatenated full y.
(d)	Local rows are independent	After x is available, each rank computes its rows with little communication.

Question 6 — Coalescing

Mapping A is preferred. Holding threadIdx.y fixed inside a warp fragment, increasing threadIdx.x by 1 increases col by 1, so the linear index row × W + col also increases by 1. In Mapping B, x-lanes change row, so addresses differ by W elements and are strided.

Question 7 — Shared-memory synchronization

The load into s and the neighbor read have a read-after-write race. A thread may read before its neighbor has stored. Insert a block barrier after every thread writes:

```
__shared__ float s[256];
int t = threadIdx.x;
int i = blockIdx.x * blockDim.x + t;
s[t] = in[i];
__syncthreads();
out[i] = s[(t + 1) % blockDim.x];
```

All block threads must reach this barrier. The barrier both waits for the block and makes the earlier shared-memory writes visible to those threads.

Question 8 — Grid-wide synchronization

No. __syncthreads() covers one block, and some blocks may not even be resident together. End the first kernel, then launch a second kernel that consumes the completed partial results. A kernel boundary provides the required global phase separation. Cooperative groups can support a grid barrier only under special cooperative-launch constraints.

Question 9 — Race and reduction

The read-modify-write sequence is not atomic: multiple threads can read the same old sum and overwrite each other's updates.

- Repair 1: use atomicAdd(&sum[0], A[i]). Correct, but every thread contends for one location.
- Repair 2: perform a hierarchical reduction—combine values within each block, then issue one atomic update per block or launch another reduction phase. This normally scales better.

MOCK TEST 3 — WORKED SOLUTIONS

Question 1 — True / False

	Ans.	Reason
(a)	T	Speedup compares serial and parallel elapsed times.
(b)	T	pT(p) is total processor-time; subtract useful serial work.
(c)	T	The bandwidth ceiling is the lower ceiling in that case.
(d)	F	Dynamic scheduling usually adds runtime assignment overhead.
(e)	F	MPI processes normally have separate address spaces.
(f)	T	OpenMP threads normally share process memory.
(g)	F	Large/spilled local objects are commonly stored in device memory, though cached.
(h)	F	Unified Memory simplifies access but migration and faults can cost time.
(i)	F	Divergence is defined within a warp, not between warps.
(j)	T	The active mask enables lanes for the current instruction/path.
(k)	T	The scheduler may interleave independent blocks.
(l)	F	An atomic protects one update; it does not wait for all threads.
(m)	T	Wavefront is the analogous AMD execution-group term.
(n)	T	Coalescing combines lane memory requests into efficient transactions.

Question 2 — Definitions

2(a) Operational intensity

The number of useful operations performed per byte transferred from the memory level being modeled, typically FLOP/byte. It determines the slope-position on a Roofline plot.

2(b) All-gather

Every rank contributes a local segment, and every rank receives the concatenation of all segments. It gathers to all participants rather than one root.

2(c) GPU texture cache/path

A read-only access path designed to exploit spatial locality, especially in 2D patterns; texture hardware may also support addressing and filtering operations.

Question 3 — Performance metrics

S = 120/20 = 6×; E = S/p = 6/8 = 0.75 = 75%; overhead = 8(20) - 120 = 40 s.

Question 4 — Scheduling irregular work

Choose dynamic scheduling because vertex costs vary greatly and a static equal-count partition can leave some threads with much more work. Dynamic assignment improves balance but incurs queue/scheduling overhead. Smaller chunks improve balance and responsiveness; larger chunks reduce overhead but can reintroduce imbalance. A moderate chunk or guided schedule is a reasonable refinement.

Question 5 — Matrix-addition launch

```
int col = blockIdx.x * blockDim.x + threadIdx.x;
int row = blockIdx.y * blockDim.y + threadIdx.y;
dim3 grid((750 + 15) / 16, (1000 + 15) / 16);  // (47, 63)
if (row < 1000 && col < 750) {
    int idx = row * 750 + col;
    C[idx] = A[idx] + B[idx];
}
```

The x dimension follows columns and the y dimension follows rows. This also gives unit-stride accesses across neighboring x-lanes in row-major storage.

Question 6 — Two adjacent outputs per thread

Correct: (a). Let B = blockDim.x and k = blockIdx.x. The first index is i = 2kB + lane, and the second is i + B.

Lane	Option (a): first access	Option (b): first access
0	2kB	2kB
1	2kB + 1	2kB + 2

Lane	Option (a): first access	Option (b): first access
2	2kB + 2	2kB + 4
3	2kB + 3	2kB + 6

Option (a) is unit stride across lanes and therefore coalesces naturally. Option (b) assigns adjacent outputs to each thread but makes the warp's simultaneous first accesses stride by two. Here 'two adjacent outputs' refers to the two contiguous B-sized sections assigned to the block; each thread handles corresponding positions i and i+B.

Question 7 — Divergence by lane pattern

The true lanes are 0, 4, 8, 12, 16, 20, 24, 28: 8 lanes. The other 24 lanes take the false path. The warp diverges and normally executes the true path with an 8-lane active mask and the false path with a 24-lane mask, then reconverges.

Question 8 — Coherence and false sharing

Each write requires exclusive ownership of the entire cache line. Because both counters occupy that line, ownership repeatedly moves between cores and each write invalidates the other core's copy. The program is logically race-free but coherence traffic serializes progress.

- Pad or align each per-thread counter so frequently written counters occupy different cache lines.
- Accumulate in thread-private variables and combine only occasionally or at the end.

Question 9 — OpenMP and MPI correctness

Part	Answer	Reason
(a)	reduction(+:sum)	Each thread accumulates privately; OpenMP combines the partial sums safely.
(b)	MPI_Allgather	All ranks contribute a segment and all ranks receive the full concatenation.

FINAL CHECKLIST

Can you derive Amdahl and Roofline results with units? Can you map x→column and y→row? Can you trace lane addresses to test coalescing? Can you separate coherence from consistency and a barrier from an atomic? If yes, you are targeting the main reasoning patterns in this exam scope.